SUMMER TRAINING REPORT

on

FUNDAMENTALS OF DATA STRUCTURES USING C++

Mini Project: File System Directory Visualizer

(Term Aug-Dec 2026)

Submitted by

Rishikesh Boppa

Registration Number: 12412603

School of Computer Science and Engineering

DECLARATION

I, Rishikesh Boppa, Registration Number [12412603], hereby declare that the Summer Training Project entitled "FILE SYSTEM DIRECTORY VISUALISER USING C++" developed during the training is an original work carried out by me under the guidance of my faculty mentor at Lovely Professional University (LPU). This project has been submitted in partial fulfilment of the requirements for the Summer Training course in Fundamentals of Data Structures Using C++.

I further declare that this report has not been submitted elsewhere for the award of any other degree, diploma, or certificate, and that all sources of information used have been duly acknowledged.

Date: 17 July 2026

Rishikesh Boppa
Registration Number: 12412603

CERTIFICATE

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty mentor for providing valuable guidance, continuous encouragement, and constructive feedback throughout the Summer Training. Their support and suggestions played an important role in the successful completion of my project entitled **"File System Directory Visualiser Using C++"**.

I am also thankful to **Lovely Professional University (LPU)** for providing the opportunity, learning resources, and training environment required to successfully complete this Summer Training project under the course **Fundamentals of Data Structures Using C++**.

I would also like to thank the faculty members of the School of Computer Science and Engineering for their support and encouragement during the training.

Finally, I express my heartfelt gratitude to my parents, family, and friends for their constant motivation, support, and encouragement throughout the completion of this project.

Rishikesh Boppa

TABLE OF CONTENTS

1. INTRODUCTION OF ORGANISATION

The Summer Training program titled “Fundamentals of Data Structures Using C++” was undertaken at Lovely Professional University (LPU) itself, as part of the university's structured Summer Term training initiative for Computer Science and Engineering students.

Lovely Professional University (LPU), located in Phagwara, Punjab, India, is one of the largest private universities in India, offering undergraduate, postgraduate, and doctoral programs across a wide range of disciplines including Engineering, Management, Law, and Sciences. The School of Computer Science and Engineering at LPU conducts industry-aligned Summer Training programs during the academic term break to help students strengthen their programming and problem-solving foundations through structured, mentor-led content delivery combined with self-learning modules and continuous assessments.

This training was delivered under the mentorship of Mr. Gaurav Sharma and focused on building strong fundamentals in the C++ programming language and Data Structures & Algorithms (DSA), which were subsequently applied to design and implement a real-world console-based mini project — the “Bank Management System” — described in detail in the following chapters.

2. SUMMER TRAINING COURSE CONTENT DETAIL

Course Name: Fundamentals of Data Structures Using C++ (Data Structures and Algorithm Design using C++)

Resource Person: Mr. Gaurav Sharma

Total Duration: 90 Hours (50 Hours Content Delivery + 40 Hours Self-Learning)

The training was structured over five weeks, combining instructor-led sessions with self-paced learning and two Continuous Assessments (CA1 and CA2). Table 1 below summarizes the week-wise breakdown of the topics covered during the training.

Table 1: Week-wise Training Schedule

Week	Topic	Content Covered	Hours
Week 1	Foundations of C++ for DSA	Introduction to logic building in programming with C++ in the DSA context; structure of a C++ program; data types & memory representation; control structures (if, loops); functions & recursion basics; time & space complexity (introduction); inheritance (types and modes), polymorphism (compile-time & run-time), and	10
Week 2	Complexity Analysis, Recursion, Arrays & Strings	Asymptotic notations (Big-O, Ω, Θ); best, worst, and average case analysis; recursive thinking and stack behaviour; solving recurrence relations; backtracking basics; static vs dynamic arrays; array operations (insert, delete, traverse); multidimensional arrays; string handling in C++; problem solving using sliding window	10
Week 3	Linked Lists & Stacks	Introduction to linked lists; singly, doubly, and circular linked list implementation; operations (insertion, deletion, traversal); applications (polynomial representation, memory management); stack implementation using arrays and linked lists; applications of stacks (expression evaluation, recursion).	10
Week 4	Data Structures and Their Applications	Stack operations (push, pop, peek) and applications (expression evaluation, backtracking); queue types (simple, circular, deque, priority queue) with implementation and real-world applications; introduction to trees, binary trees and binary search trees (BST); tree traversal methods (pre-order, in-order, post-order); AVL trees (introduction and rotations); graph basics, directed vs undirected graphs, graph	10
Week 5	Introduction to Algorithms	Searching algorithms — linear, binary, and interpolation search; sorting algorithms — selection sort and bubble sort; algorithm design techniques — greedy algorithms, divide & conquer, and dynamic programming basics. Includes Continuous Assessment 2 (CA2) — Project.	10

SELF-LEARNING RESOURCES

In addition to the instructor-led sessions, 40 hours were allocated for self-learning using the following resources:

- C++ Learning: learncpp.com, w3schools.com/cpp, GeeksforGeeks C++, TutorialsPoint C++, freeCodeCamp C++ Course (YouTube).
- Data Structures & Algorithms: w3schools.com/dsa, TutorialsPoint DSA, GeeksforGeeks Data Structures, Programiz DSA, VisuAlgo.
- Logic Building for DSA: GeeksforGeeks and freeCodeCamp articles on building strong DSA fundamentals.
- Practice Platforms: LeetCode, HackerRank, CodeChef, GUVI Code Kata, TechieDelight, Edabit.

3. SUMMER TRAINING PROJECT DETAIL

3.1 PROBLEM STATEMENT

Managing and understanding a computer's file system becomes increasingly difficult as the number of files and folders grows. Traditional file explorers display directory structures, but they do not clearly represent the hierarchical relationship between folders or provide the cumulative size occupied by each directory. Manually calculating folder sizes and understanding deeply nested structures is time-consuming and prone to errors.

The objective of this project is to develop a File System Directory Visualiser using C++, N-ary Trees, and Recursion. The application recursively scans a given directory, builds a tree representation of the file system, calculates the total size of each folder, and displays the directory hierarchy in a clear and readable format through the console.

3.2 PROJECT OUTCOMES

The main objectives of the File System Directory Visualiser are:

- To recursively scan the contents of a directory and all its subdirectories.
- To represent the file system using an **N-ary Tree** data structure.
- To calculate the cumulative size of every directory using **post-order traversal**.
- To display the directory hierarchy in a structured tree format.
- To demonstrate the practical application of recursion and tree data structures.
- To provide an organized and easy-to-understand visualisation of folder structures.
- To improve understanding of file system organisation using Data Structures concepts.

3.3 TECHNOLOGIES USED

The following technologies were used in developing this project:

- **Programming Language:** C++ (C++17 Standard)
- **Data Structure:** N-ary Tree
- **Algorithm:** Recursive Tree Traversal (Depth-First Search)
- **File Handling:** C++ `<filesystem>` library for scanning directories and retrieving file information.
- **Standard Template Library (STL):** Used for vectors, strings, and other utility classes.
- **Development Environment:** Visual Studio Code
- **Compiler:** Apple Clang++ (C++17)
- **Operating System:** macOS

4. SOURCE CODE

The project has been developed using a modular programming approach in C++. The source code is organized into multiple header (.h) and implementation (.cpp) files, making the program easier to understand, maintain, and extend.

The project consists of the following modules:

- **Node:** Represents each file or directory in the N-ary Tree.
- **TreeBuilder:** Recursively scans the file system and constructs the tree.
- **SizeCalculator:** Calculates the cumulative size of directories using post-order traversal.
- **Visualiser:** Displays the complete directory hierarchy in a tree-like format.
- **Main Program:** Coordinates the execution of all modules and generates the final output.

The complete source code has been submitted along with this report as part of the project files.

BIBLIOGRAPHY

1. Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley, 2013.
2. CppReference. *C++ Standard Library Documentation*. https://en.cppreference.com
3. LearnCpp. *Free C++ Programming Tutorial*. https://www.learncpp.com
4. GeeksforGeeks. *Data Structures and Algorithms using C++*. https://www.geeksforgeeks.org
5. Microsoft Learn. *C++ Documentation*. https://learn.microsoft.com/cpp